



UNIVERSIDAD AUTÓNOMA DE GUERRERO

FACULTAD DE INGENIERÍA



Programa Educativo:

Ingeniero en Computación

Unidad de Aprendizaje:

Ingeniería de Software

Profesor: Ruben Rodriguez Camargo

Alumno: Araceli Medel Carranza

Lugar Y Fecha:

Chilpancingo De Los Bravo,
Gro A 23 De Febrero Del 2026

INDICE

1.	-----	CICLO DE VIDA DEL SOFTWARE
2.	-----	Fases Principales del SDLC
3.	-----	Los Principales Modelos del SDLC
3.1	-----	Modelo en Cascada (Waterfall)
3.2	-----	Modelo Ágil (Agile)
3.3	-----	Modelo Iterativo
3.4	-----	Modelo en Espiral
3.5	-----	Modelo en V (Verificación y Validación)
3.6	-----	Modelo RAD (Desarrollo Rápido de Aplicaciones)
4.	-----	Cómo Elegir el Modelo Correcto
4.1	-----	Guía práctica de selección
5.	-----	En Resumen
6.	-----	Fuentes Consultadas

CICLO DE VIDA DEL SOFTWARE

1. ¿Qué es el Ciclo de Vida del Software?

Cuando se habla del Ciclo de Vida del Desarrollo de Software, conocido mundialmente por sus siglas en inglés SDLC (Software Development Life Cycle), se hace referencia a un proceso organizado y sistemático que guía a los equipos de desarrollo desde el primer día de un proyecto hasta el momento en que el sistema deja de operar. No es simplemente una lista de pasos: es un marco de trabajo completo que permite planificar, controlar y entregar software de calidad de manera ordenada, controlando tanto los costos como los tiempos de entrega.

Para entenderlo de forma sencilla, el SDLC funciona como una hoja de ruta. Así como un arquitecto no construye un edificio sin planos, un equipo de desarrollo de software no puede crear un sistema serio sin un proceso claro que defina qué se hará, cuándo y cómo. El SDLC cumple exactamente esa función: pone orden en un proceso que, sin estructura, puede volverse caótico y costoso.

2. Fases Principales del SDLC

El SDLC se divide en fases o etapas bien definidas. Aunque el número exacto puede variar según el autor o la metodología adoptada, la mayoría de las fuentes consultadas coinciden en siete etapas fundamentales que recorre todo proyecto de software:

Nº	Fase	Descripción resumida
1	Planificación	Se define el alcance del proyecto, se analiza su viabilidad técnica y económica, y se elabora el plan general.
2	Análisis de Requisitos	Se recogen todas las necesidades del cliente y se documentan de forma precisa para guiar el desarrollo.
3	Diseño	Se establece la arquitectura del sistema: cómo se organizan los datos, los módulos y las interfaces de usuario.
4	Implementación	El equipo de desarrollo escribe el código del sistema basándose en los diseños aprobados.
5	Pruebas	Se verifica que el software funcione correctamente, sin errores, y cumpla con todos los requisitos definidos.
6	Despliegue	El sistema aprobado se pone en producción y los usuarios finales comienzan a utilizarlo.
7	Mantenimiento	Se corrigen errores post-lanzamiento, se mejora el rendimiento y se añaden nuevas funcionalidades con el tiempo.

La fase más crítica de todo el proceso es, sin duda, el análisis de requisitos. Como señala Isvisoft, es aquí donde los miembros más experimentados del equipo recopilan la información que da el cliente, el departamento de ventas, marketing y expertos del dominio. Si esta etapa se hace mal, todo lo que viene después estará construido sobre bases equivocadas, lo que inevitablemente llevará a rehacer trabajo o, en el peor caso, a entregar un producto que nadie quiere usar.

3. Los Principales Modelos del SDLC

Una de las decisiones más importantes al iniciar un proyecto de software es elegir con qué modelo se va a trabajar. Un modelo del SDLC es la manera en que se organiza y secuencía el proceso de desarrollo. No todos los proyectos son iguales: algunos tienen requisitos muy claros desde el inicio, otros son exploratorios y cambian constantemente. Por eso existen varios modelos, cada uno pensado para un tipo diferente de situación.

3.1 Modelo en Cascada (Waterfall)

Modelo en Cascada (Waterfall)

Es el modelo más antiguo y el primero en formalizarse. Su funcionamiento es simple: las fases se ejecutan una tras otra en orden estricto, como el agua que cae por una cascada. Primero se planifica, luego se analiza, luego se diseña, y así sucesivamente. Ninguna fase comienza hasta que la anterior ha sido completada y aprobada formalmente. Esto le da una estructura muy clara y predecible al proyecto. Sin embargo, su mayor debilidad es la rigidez. En el mundo real, los requisitos cambian, los clientes ajustan sus ideas y aparecen tecnologías nuevas. Con el modelo Cascada, incorporar esos cambios es muy costoso porque implica retroceder a etapas ya cerradas.

3.2 Modelo Ágil (Agile)

Modelo Ágil (Agile)

El modelo Ágil nació como respuesta directa a las limitaciones del Cascada. En lugar de planificar todo desde el principio y avanzar de forma lineal, el desarrollo ágil se organiza en ciclos cortos llamados iteraciones o sprints, que normalmente duran entre una y cuatro semanas. Al final de cada ciclo se entrega una versión funcional del producto, aunque parcial. Esta forma de trabajar permite que el cliente vea avances frecuentes y pueda dar retroalimentación real, no solo al final del proyecto sino durante todo el proceso. Si algo no le gusta o cambian sus necesidades de negocio, el equipo puede ajustarse en el siguiente sprint sin que eso signifique tirar todo lo hecho.

3.3 Modelo Iterativo

Modelo Iterativo

El modelo iterativo funciona construyendo el software en ciclos repetitivos. En cada ciclo o iteración, el equipo toma un subconjunto de requisitos, los diseña, los codifica y los prueba. El resultado es una versión parcial del sistema que mejora con cada vuelta hasta que el producto final está listo. A diferencia del Ágil puro, el iterativo no necesariamente involucra al cliente en cada ciclo de la misma manera. Su ventaja principal es que permite comenzar con requisitos incompletos o poco definidos y ir refinándolos conforme avanza el desarrollo. Es especialmente útil cuando el equipo necesita aprender sobre el dominio del problema mientras construye la solución.

3.4 Modelo en Espiral

Modelo en Espiral

Propuesto por Barry Boehm en 1988, el modelo en espiral es una combinación entre el enfoque estructurado del Cascada y el enfoque iterativo del prototipado. Lo que lo distingue de todos los demás es que coloca el análisis de riesgos en el centro de cada vuelta del espiral. Cada ciclo atraviesa cuatro etapas: definir los objetivos, identificar y resolver los riesgos técnicos o de negocio, desarrollar y probar, y finalmente evaluar los resultados con el cliente antes de planificar la siguiente vuelta. Esta atención constante a los riesgos lo convierte en el modelo ideal para proyectos grandes, complejos, o que involucren tecnología poco probada.

3.5 Modelo en V (Verificación y Validación)

V Modelo en V

El modelo en V es una extensión del Cascada que corrige uno de sus problemas principales: las pruebas solo al final. En el modelo en V, cada fase de desarrollo tiene una fase de prueba planeada en paralelo. Visualmente, el lado izquierdo de la V representa las fases de construcción y el lado derecho las de verificación, convergiendo ambos lados en la codificación. Por ejemplo, cuando se realiza el diseño del sistema, simultáneamente se planifican las pruebas de sistema que se ejecutarán más adelante. Esto garantiza que las pruebas estén alineadas con lo que se construyó, mejorando notablemente la calidad final del producto.

3.6 Modelo RAD (Desarrollo Rápido de Aplicaciones)

Modelo RAD – Rapid Application Development

El RAD prioriza la velocidad sobre la perfección inicial. Utiliza herramientas de generación de código, componentes reutilizables y sesiones intensivas de trabajo conjunto con el cliente (conocidas como JAD sessions) para entregar un prototipo funcional en tiempos muy cortos, típicamente entre 60 y 90 días. Es muy útil cuando el tiempo es el factor más crítico del proyecto. Sin embargo, requiere equipos muy capacitados y un cliente comprometido que esté disponible para dar retroalimentación constante. Si alguna de estas condiciones falta, el modelo puede generar código de baja calidad y una arquitectura deficiente.

4. ¿Cómo Elegir el Modelo Correcto?

Después de revisar los distintos modelos, surge la pregunta natural: ¿cuál es el mejor? La respuesta, según todas las fuentes consultadas, es que no existe un modelo universalmente superior. La elección depende siempre del contexto específico del proyecto. Una aplicación bancaria con requisitos regulatorios estrictos no se desarrolla igual que una startup que necesita validar su idea en el mercado en pocas semanas.

4.1 Guía práctica de selección

Con base en lo investigado, se puede resumir la selección del modelo en función de tres variables principales del proyecto:

Si los requisitos son...	Y el proyecto es...	Modelo recomendado
Fijos y bien documentados	Grande con riesgo técnico alto	Cascada o Modelo en V
Claros pero puede haber riesgos	Complejo e innovador	Espiral
Cambiantes o poco definidos	Mediano con entregas frecuentes	Ágil (Scrum/Kanban)
Parcialmente definidos	Cualquier tamaño	Iterativo
Urgentes, fecha límite corta	Pequeño a mediano	RAD

5. En Resumen

Después de revisar múltiples fuentes sobre el tema, puedo decir que el Ciclo de Vida del Software es mucho más que una serie de pasos técnicos: es la diferencia entre hacer software de forma improvisada y hacerlo de manera profesional.

Antes de conocer este tema, uno podría pensar que programar es simplemente sentarse a escribir código. Pero la realidad es que sin planificación, sin un análisis claro de lo que se necesita, sin un diseño previo de la arquitectura y sin pruebas sistemáticas, lo que se obtiene es un producto deficiente, fuera de presupuesto y que no resuelve el problema real del cliente. El SDLC existe precisamente para evitar eso.

Lo más interesante que encontré en esta investigación es que no hay un solo camino correcto. Existen múltiples modelos porque existen múltiples tipos de proyectos y equipos. Un proyecto gubernamental con requisitos legales fijos se beneficia del Cascada por su orden y su documentación. Una startup tecnológica que necesita iterar rápido con su cliente

se beneficia del Ágil por su flexibilidad. Un sistema de control para un avión necesita el rigor del Modelo en V para garantizar que nada falle. Cada modelo tiene su razón de existir.

También me pareció relevante entender que el mantenimiento, aunque suele mencionarse al final como si fuera algo secundario, representa en la práctica entre el 60% y el 80% del costo total de un proyecto de software. Esto quiere decir que el ciclo de vida no termina cuando se lanza el producto: en cierto modo, apenas comienza. Mantener el software actualizado, seguro y funcionando bien es un trabajo continuo que puede durar años.

En conclusión, el SDLC no es un conjunto de reglas burocráticas para complicar el trabajo de los desarrolladores. Es una herramienta de gestión que, cuando se aplica bien, ayuda a entregar mejores productos, en menos tiempo, con menos errores y con mayor satisfacción tanto para el equipo que lo construye como para el cliente que lo usa.

Fuentes Consultadas:

#	Fuente / Autor	Sitio	URL
1	Amazon Web Services (AWS)	aws.amazon.com	https://aws.amazon.com/es/what-is/sdlc/
2	Isvisoft – Equipo editorial	isvisoft.com	https://isvisoft.com/ciclo-vida-software-sdlc/
3	Visure Solutions	visuresolutions.com	https://visuresolutions.com/es/alm-guide/systems-development-life-cycle-models/
4	Futurum Technology	futurum.tech	https://www.futurum.tech/blog/index.php/2024/04/16/modelos-del-ciclo-de-vida-del-desarrollo-de-software/
5	Geekflare España	geekflare.com/es	https://geekflare.com/es/software-development-life-cycle-sdlc-guide/